ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

TRẦN MẠNH DUY

XÂY DỰNG ĐƯỜNG CONG ELLIPTIC BẬC SIÊU CAO VÀ ỨNG DỤNG CHỮ KÝ SỐ
KHÓA LUẬN TỐT NGHIỆP

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Công nghệ thông tin định hướng thị trường Nhật Bản

HÀ NỘI - 2026

ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ

TRẦN MẠNH DUY

XÂY DỰNG ĐƯỜNG CONG ELLIPTIC BẬC SIÊU CAO VÀ ỨNG DỤNG CHỮ KÝ SỐ

KHÓA LUẬN TỐT NGHIỆP

KHÓA LUẬN TỐT NGHIỆP ĐẠI HỌC HỆ CHÍNH QUY

Ngành: Công nghệ thông tin định hướng thị trường Nhật Bản

Cán bộ hướng dẫn: GS. TS. Lê Phê Đô

HÀ NỘI - 2025

LỜI CAM ĐOAN

Tôi xin cam đoan khóa luận tốt nghiệp ***Xây dựng đường cong Elliptic bậc siêu cao và ứng dụng Chữ ký số*** là do tôi tự tìm hiểu, nghiên cứu và phát triển dưới sự hướng dẫn của GS. TS. Lê Phê Đô, không sao chép hay sử dụng bất kỳ phần nào từ các công trình nghiên cứu của người khác mà không trích dẫn nguồn theo quy định.. Tất cả những tài liệu tham khảo đều đã được liệt kê ở phần Tài liệu tham khảo của khóa luận. Tôi xin hoàn toàn chịu trách nhiệm về tính trung thực và chính xác của toàn bộ nội dung trong khóa luận này.

Tác giả khóa luận

Trần Mạnh Duy

LỜI CẢM ƠN

Viết lời cảm ơn ở đây…

MỤC LỤC

Lời cam đoan . . . . . .
Lời cảm ơn . . . . . .
Mục lục . . . . . **i**
Tóm tắt . . . . . **iv**
Danh mục viết tắt . . . . . **v**
Danh mục hình vẽ . . . . . **vi**
Danh mục bảng biểu . . . . . **vii**
Mở đầu **1**
Chương 2. Tổng quan và Giới thiệu bài toán **4**
2.0.1. Tổng quan về Hệ mật mã khóa công khai và ECC 4
2.0.2. Thực trạng các tiêu chuẩn ECC và Vấn đề tự chủ mật mã 4
2.0.3. Vai trò của ngôn ngữ lập trình trong an toàn mật mã 5
2.0.4. Phát biểu bài toán và Phạm vi nghiên cứu 6
Chương 3. Cơ sở toán học **8**
3.1. Định nghĩa Trường số nguyên tố 8
3.1.1. Cài đặt: `PrimeFieldConfig` và `PrimeFieldElement` . . . 8
3.1.2. Các phép toán cơ bản 9
3.2. Đường cong elliptic Short Weierstrass 9
3.2.1. Định nghĩa 9
3.2.2. Cấu trúc nhóm 10
3.3. Phép cộng điểm (Point Addition) 10
3.3.1. Quy tắc hình học 10
3.3.2. Công thức đại số 10
3.3.3. Cài đặt 11
3.4. Phép nhân vô hướng (Scalar Multiplication) 11
3.4.1. Thuật toán Double-and-Add 11
3.4.2. Cài đặt 12
3.4.3. Tầm quan trọng 12
3.5. Tối ưu số học: Phép nhân Montgomery 13

3.5.1. Vấn đề 13
3.5.2. Biến đổi Montgomery 13
3.5.3. Chuyển đổi vào/ra không gian Montgomery 13
3.6. Lũy thừa nhanh (Square-and-Multiply) 14
Chương 4. Phương pháp xây dựng đường cong elliptic 15
4.1. Ý tưởng cốt lõi 15
4.1.1. Điều kiện CM và phương trình Diophantine 15
4.1.2. Đa thức lớp Hilbert $H_D(x)$ 16
4.1.3. Quy trình CM đầy đủ 16
4.1.4. Trường hợp $D = -3$: Xoắn sextic 17
4.2. Tham số đầu vào của hệ thống 17
4.2.1. `k = 18` — Bậc nhúng (Embedding Degree) 18
4.2.2. `d = 3` — Biệt thức CM ($|D|$) 18
4.2.3. `target_r_bits = 512` — Kích thước bậc nhóm r 18
4.2.4. `target_p_bits = 1024` — Kích thước trường cơ sở p 19
4.2.5. `max_attempts = 100_000` — Số lần thử tối đa 19
4.2.6. `min_scalar_two_adicity = 32` — Two-adicity tối thiểu của $r - 1$ 19
4.2.7. `min_base_two_adicity = 32` — Two-adicity tối thiểu của $p - 1$ 20
4.2.8. Tổng hợp: Đặc trưng đường cong mục tiêu 20
4.3. Thuật toán Cocks-Pinch truyền thống 20
4.3.1. Vấn đề: Xây dựng ngược từ r 20
4.3.2. Điều kiện bậc nhúng 21
4.3.3. Thuật toán CP cho $k = 18$ 21
4.3.4. Phân tích độ phức tạp trung bình 22
4.4. Thuật toán Cocks-Pinch Cải tiến 22
4.4.1. Động lực: Tấn công rây trường số tháp (TNFS) 22
4.4.2. Yêu cầu NTT-friendly 23
4.4.3. Cải tiến 1: NTT-friendly r qua ràng buộc T 23
4.4.4. Cải tiến 2: NTT-friendly p qua mở rộng lưới nâng 24
4.4.5. Phân tích hiệu năng của thuật toán cải tiến 25
4.4.6. So sánh với BLS12-381 25

4.4.7. Giới hạn và hướng mở rộng 26
Chương 5. SƠ ĐỒ CHỮ KÝ SỐ 27
5.1. Cặp khóa (KeyPair) 27
5.1.1. Hàm băm mở rộng 28
5.2. Chữ ký Schnorr 28
5.2.1. Lý thuyết 28
5.2.2. Cấu trúc dữ liệu 29
5.2.3. Cài đặt 29
5.2.4. Bảo mật 29
5.3. Chữ ký ECDSA 30
5.3.1. Lý thuyết 30
5.3.2. Cấu trúc dữ liệu 30
5.3.3. Cài đặt 30
5.3.4. So sánh Schnorr và ECDSA 31
5.4. Chữ ký tổng hợp BLS 31
5.4.1. Lý thuyết (tổng quan) 31
5.4.2. Vai trò của đường cong KSS18 32
5.5. Cài đặt hệ thống ký (`curve1024-sig`) 32
Chương 6. CÀI ĐẶT VÀ ĐÁNH GIÁ 34
6.0.1. Đánh giá hiệu năng 34
6.0.2. Đánh giá độ an toàn 35
6.0.3. Kết luận và hướng phát triển 37
Chương 7. KẾT LUẬN 39
7.0.1. Tổng kết 39
7.0.2. Hạn chế 39
7.0.3. Hướng phát triển 39
TÀI LIỆU THAM KHẢO 40

TÓM TẮT

Viết tóm tắt nội dung khóa luận ở đây…

Từ khóa: từ khóa 1, từ khóa 2, từ khóa 3

DANH MỤC VIẾT TẮT

Viết tắt	Giải thích
AI	Artificial Intelligence (Trí tuệ nhân tạo)
ML	Machine Learning (Học máy)
API	Application Programming Interface

DANH MỤC HÌNH VẼ

DANH MỤC BẢNG BIỂU

MỞ ĐẦU

Điện toán hiện đại đang phụ thuộc sâu sắc vào mã hóa khóa công khai, trong đó Đường cong Elliptic (Elliptic Curve Cryptography - ECC) đóng vai trò xương sống cho hầu hết các giao thức bảo mật từ TLS, SSH cho đến các hệ thống blockchain. Tuy ECC mang lại hiệu năng và kích thước khóa tối ưu, việc phụ thuộc vào các thư viện và cấu hình sinh sẵn từ các tổ chức nước ngoài cũng đặt ra những thách thức lớn về an ninh và làm chủ công nghệ lõi. Việc tự xây dựng một hệ thống từ tầng thấp nhất (Lớp số nguyên lớn U1024) đến tầng cao nhất (Chữ ký Schnorr/ECDSA) giúp loại bỏ cái vỏ bọc "hộp đen" công nghệ.

0.0.0.1. Lý do chọn đề tài

Sự cần thiết của Tự chủ mật mã (Loại bỏ "Hộp đen" công nghệ): Hiện nay, hầu hết các hệ thống phần mềm đều sử dụng thiết kế các đường cong do Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST) hoặc các tổ chức nước ngoài khuyến cáo (như secp256r1 hay ed25519) và đóng gói sẵn dưới dạng thư viện "hộp đen" (ví dụ: OpenSSL). Tuy nhiên, việc phụ thuộc hoàn toàn vào các bộ tham số sinh sẵn đã dấy lên nhiều lo ngại về an ninh quốc gia, điển hình là nghi án cơ quan tình báo cài cắm "cửa hậu" (backdoor) vào thuật toán sinh số ngẫu nhiên Dual_EC_DRBG [1]. Việc tự xây dựng một hệ thống giúp chúng ta có khả năng kiểm duyệt và làm chủ hoàn toàn công nghệ lõi, đảm bảo xác suất bằng không đối với bất kỳ dòng mã độc ngầm hay cửa hậu nào. Đây là bước đệm cần thiết hướng tới việc tự chủ "đường cong thuần Việt".

Nhu cầu an toàn tuyệt đối trước sức mạnh máy tính cổ điển: Hiện nay, các tiêu chuẩn an toàn thông tin do NIST khuyến nghị thường chỉ định Đường cong Elliptic ở mức 256-bit đến tối đa 521-bit [2]. Tuy đủ dùng trong bối cảnh hiện tại nhưng khóa 256-bit chỉ cung cấp độ an toàn dài hạn (khoảng 128-bit an toàn đối xứng). Đề tài này nhằm khám phá khả năng và sự đánh đổi (trade-off) khi đẩy cấp độ bảo mật lên bậc cực hạn: đường cong trên trường nguyên tố 1024-bit. Với tham số khổng lồ này, hệ thống cung cấp mức độ an toàn xấp xỉ 512-bit, khiến cho mọi cuộc tấn công bằng máy tính cổ điển sử dụng các thuật toán giải bài toán logarit rời rạc (DLP) tốt nhất hiện nay như thuật toán Pollard's rho hay Baby-step Giant-step trở thành bất khả thi tuyệt đối trong

biên niên kỷ tương lai.

Vấn đề an toàn bộ nhớ trong Mật mã học: Trong thực tế triển khai, các thư viện mật mã truyền thống chủ yếu được viết bằng C/C++, dẫn tới việc thường xuyên đối mặt với các lỗ hổng hệ thống nghiêm trọng liên quan đến an toàn bộ nhớ. Tiêu biểu có thể kể đến sự cố rò rỉ bộ nhớ Heartbleed năm 2014, đe dọa trực tiếp sự toàn vẹn của nền tảng Internet toàn cầu [3]. Theo thống kê từ báo cáo của hệ điều hành Microsoft, khoảng 70% các bản vá lỗi bảo mật (CVE) cốt lõi của họ đều có nguyên nhân trực tiếp từ những lỗi quản lý bộ nhớ như vậy [4]. Thiết kế của đề tài này giải quyết triệt để rủi ro đó ngay ở giai đoạn đầu bằng cách sử dụng Rust – một ngôn ngữ lập trình hệ thống hiện đại, đảm bảo nguyên tắc an toàn bộ nhớ (memory-safe) ở cấp độ trình biên dịch. Điều này giúp hệ thống triệt tiêu hoàn toàn rủi ro tràn bộ nhớ đệm (buffer overflow) hay sử dụng vùng nhớ đã giải phóng (use-after-free) phổ biến trong các thư viện tiền nhiệm.

0.0.0.2. Đóng góp của đề tài

Trong khóa luận này, hệ thống thiết kế tập trung nghiên cứu, thiết kế và cài đặt một nền tảng mật mã đường cong elliptic với tham số kích thước cực lớn (1024-bit). Khóa luận đạt được các đóng góp sau: - Xây dựng từ đầu hệ thống số học bignum cho cấp độ phần cứng bằng ngôn ngữ hệ thống đảm bảo an toàn bộ nhớ và luồng thực thi tuyệt đối. - Ứng dụng thành công lý thuyết thuật toán CM (Complex Multiplication) và phương pháp Cocks-Pinch cải tiến để chủ động sinh ra một đường cong “pairing-friendly” mới 1024-bit không có trong bất kỳ tiêu chuẩn có sẵn nào, nhưng vẫn đảm bảo đặc tính bảo mật chống các cuộc gọi tấn công MOV và Anomalous attack nổi tiếng. - Cài đặt và tích hợp ba sơ đồ chữ ký tiêu chuẩn: ECDSA, Schnorr và chữ ký tổng hợp nền tảng ghép cặp BLS.

0.0.0.3. Bố cục của khóa luận

Nội dung của khóa luận được trình bày bao gồm các phần chính sau:

- **Phần Mở đầu**: Trình bày tính cấp thiết, mục đích, nội dung và đóng góp của khóa luận.
- **Chương 1. Giới thiệu bài toán**: Khái quát phạm vi đồ án và bối cảnh sử dụng.
- **Chương 2. Cơ sở toán học**: Cung cấp các kiến thức nền tảng trong lý thuyết nhóm, trường hữu hạn, định nghĩa và luật tính toán trên đường cong elliptic.
- **Chương 3. Phương pháp xây dựng đường cong elliptic**: Trình bày lý thuyết và biến đổi toán học cùng thuật toán Cocks-Pinch để tính toán và tự thiết kế tham số

đường cong.

- **Chương 4. Sơ đồ chữ ký số**: Mô tả cấu trúc dữ liệu và logic thiết kế phần mềm bằng Rust triển khai ba thuật toán chữ ký ECDSA, Schnorr và chữ ký BLS.
- **Chương 5. Cài đặt và Đánh giá**: Tổng hợp thông số kỹ thuật thực nghiệm, báo cáo đánh giá benchmark phần cứng, chứng minh độ tin cậy và so sánh tính bảo mật thực tiễn so với các nền tảng mật mã tiêu chuẩn toàn cầu. Cuối cùng, luận văn mở ra những đề xuất định hướng khắc phục hạn chế và tối ưu quy mô sản phẩm trong tương lai.

[0] [1] D. Shumow and N. Ferguson, "On the possibility of a back door in the NIST SP800-90 dual ec prng," in *CRYPTO 2007 rump session*, 2007.

[0] [2] E. Barker, "Recommendation for key management, part 1: general," National Institute of Standards; Technology, NIST Special Publication 800-57 Part 1 Revision 5, 2020. doi: 10.6028/NIST.SP.800-57pt1r5.

[0] [3] CVE, "CVE-2014-0160: The heartbleed bug." `https://cve.mitre.org/cgi-bin/cvename.cgi?name=cve-2014-0160`, 2014.

[0] [4] M. Miller, "Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape." BlueHat IL, 2019.

CHƯƠNG 2
TỔNG QUAN VÀ GIỚI THIỆU BÀI TOÁN

Chương này cung cấp cái nhìn tổng quan về bối cảnh của Hệ mật mã khóa công khai, thực trạng của các tiêu chuẩn Đường cong Elliptic hiện hành, và phân tích những rủi ro bảo mật cốt lõi về cả mặt toán học lẫn triển khai phần mềm. Từ đó, chương làm nổi bật tính cấp thiết của bài toán xây dựng đường cong Elliptic bậc siêu cao bằng ngôn ngữ an toàn bộ nhớ.

2.0.1. Tổng quan về Hệ mật mã khóa công khai và ECC

Sự ra đời của Mật mã khóa công khai (Public Key Cryptography - PKC), đánh dấu bởi các công trình của Diffie-Hellman và RSA, là một bước ngoặt trong lịch sử an toàn thông tin. Mô hình mã hóa sử dụng khóa bất đối xứng (một khóa công khai để mã hóa/xác minh và một khóa riêng tư để giải mã/ký trí) đã giải quyết triệt để bài toán phân phối khóa qua kênh truyền không an toàn – điều mà hệ mật mã khóa đối xứng truyền thống không thể làm được.

Khi nhu cầu tính toán và trao đổi dữ liệu tăng vọt, Hệ mật mã Đường cong Elliptic (Elliptic Curve Cryptography - ECC) ra đời như một giải pháp thay thế ưu việt cho RSA. Sức mạnh vượt trội của ECC nằm ở tỷ lệ giữa kích thước khóa và độ an toàn. Để đạt được mức độ bảo mật 128-bit (mức tối thiểu được khuyến cáo hiện nay), hệ thống RSA cần sử dụng độ dài khóa lên tới 3072-bit. Trong khi đó, hệ thống ECC chỉ yêu cầu khóa dài 256-bit [1]. Kích thước khóa nhỏ hơn đáng kể giúp ECC tiêu thụ ít tài nguyên tính toán, băng thông và điện năng hơn, biến nó trở thành tiêu chuẩn vàng cho các giao thức mạng hiện đại như TLS, mạng tiền điện tử (Bitcoin, Ethereum), và các hệ thống chữ ký số điện tử.

2.0.2. Thực trạng các tiêu chuẩn ECC và Vấn đề tự chủ mật mã

Mặc dù ECC mang lại lợi ích lớn về hiệu năng, việc triển khai ECC trong thực tế lại bộc lộ một "nỗi đau" về mặt công nghệ và chủ quyền an ninh. Hiện nay, hầu hết các hệ thống phần mềm đều tái sử dụng các đường cong được định nghĩa và sinh sẵn bởi các tổ chức quốc tế, điển hình như Viện Tiêu chuẩn và Công nghệ Quốc gia Hoa Kỳ (NIST, ví dụ tập tham số P-256) hoặc Tổ chức SECG (ví dụ hàm secp256k1).

Việc sử dụng các hằng số toán học (các hệ số p, a, b) được sinh ra từ các "hạt giống" (seed) không giải thích được rõ ràng tiềm ẩn nguy cơ hệ thống bị cài cắm "cửa hậu" (backdoor). Nguy cơ này không chỉ nằm trên lý thuyết mà đã trở thành hiện thực với sự kiện cơ quan tình báo NSA bị cáo buộc can thiệp và cài backdoor vào tiêu chuẩn thuật toán sinh số ngẫu nhiên Dual_EC_DRBG [2]. Sự kiện này đặt ra yêu cầu bức thiết về năng lực **"Tự chủ mật mã"** định hướng tương lai – nơi các hệ thống quốc gia có khả năng tự chủ thiết kế, đánh giá và làm chủ các tham số đường cong bằng các phương pháp toán học minh bạch mà không cần phụ thuộc vào các thư viện "hộp đen" từ nước ngoài.

Bên cạnh đó, xét về giới hạn an toàn của máy tính cổ điển: với sự tiến bộ của phần cứng, khóa 256-bit dù vẫn an toàn ở thời điểm hiện tại nhưng chỉ đạt biên độ an toàn 128-bit. Do đó, cần có những nghiên cứu đón đầu nhằm đánh giá khả năng của các đường cong "bậc siêu cao", ví dụ 1024-bit. Khóa 1024-bit cung cấp mức độ an toàn khổng lồ, xấp xỉ 512-bit, khiến mọi cuộc tấn công bằng thuật toán tốt nhất hiện nay (như Pollard's rho) trở nên hoàn toàn vô vọng trong các giới hạn vật lý của vũ trụ.

2.0.3. Vai trò của ngôn ngữ lập trình trong an toàn mật mã

Trong lịch sử ngành an toàn thông tin, những sự cố bảo mật tồi tệ nhất của các thư viện mật mã lớn (tiêu biểu như OpenSSL) thường không xuất phát từ sai sót trong toán học lý thuyết, mà bắt nguồn từ các lỗ hổng quản lý bộ nhớ của ngôn ngữ lập trình C/C++. Các lỗi tràn bộ đệm (buffer overflow), truy cập vùng nhớ đã bị giải phóng (use-after-free) hay đọc quá giới hạn vùng nhớ đã gây ra hàng loạt thảm họa bảo mật, nổi tiếng nhất là sự cố rò rỉ bộ nhớ Heartbleed năm 2014 [3]. Thực tế triển khai một hệ thống xử lý số nguyên lớn (BigInt) có kích thước lên đến 1024-bit bằng C/C++ là một thách thức cực đoan về quản lý an toàn bộ nhớ. Theo số liệu từ quỹ phản ứng sự cố của Microsoft, có tới xấp xỉ 70% các lỗ hổng thực thi mã từ xa cấu thành từ nhóm lỗi an toàn bộ nhớ [4].

Khóa luận này khắc phục hoàn toàn điểm yếu kiến trúc trên thông qua việc ứng dụng ngôn ngữ lập trình hệ thống hiện đại **Rust**. Với cơ chế Ownership (quyền sở hữu) và quy tắc Borrow Checker (kiểm tra mượn tĩnh) nghiêm ngặt tại thời điểm biên dịch (compile-time), trình biên dịch (compiler) của Rust loại bỏ toán toàn các lỗi an toàn bộ nhớ mà không cần tới bộ thu gom rác (Garbage Collector). Cách tiếp cận này giúp thư viện mật mã đạt được tốc độ thực thi (runtime) ngang ngửa với C/C++ trong khi được bảo hành tuyệt đối về mặt bộ nhớ theo đặc tả của ngôn ngữ.

2.0.4. Phát biểu bài toán và Phạm vi nghiên cứu

Dựa trên những phân tích về lỗ hổng của thư viện mật mã truyền thống và yêu cầu về tính tự chủ, khóa luận đặt ra mục tiêu thiết kế và cài đặt từ đầu (from scratch) một thư viện mã nguồn mở cho Hệ mật mã Đường cong Elliptic với những đặc điểm cơ bản sau:

1. **Về mặt toán học**: Chủ động tính toán, phân tích và sinh mới bề mặt đường cong Elliptic trên trường nguyên tố siêu lớn **1024-bit**, tự chủ hoàn toàn các thuật toán sinh số mà không tái sử dụng hằng số của tổ chức thứ ba.
2. **Về mặt kiến trúc phần mềm**: Mã nguồn được thiết kế phân tầng theo quy tắc đơn giản hóa (Keep It Simple), chia hệ thống thành ba lớp biệt lập, thuận tiện cho việc kiểm định, bao gồm:
   - *Lớp Số nguyên lớn* (`U1024`): Tự định nghĩa kiểu dữ liệu 1024-bit với các phép lặp tối ưu phần cứng.
   - *Lớp Trường hữu hạn* (`Prime Field`): Các phép toán đại số modulo bảo mật.
   - *Lớp Tọa độ điểm* (`Affine Point`): Biểu diễn hình học trên đường cong.
3. **Về mặt ứng dụng**: Cài đặt thành công, kiểm thử tính đúng đắn và đánh giá hiệu năng đo lường đối với kiến trúc chữ ký số tiêu chuẩn (ECDSA / Schnorr) áp dụng trục tiếp lên đường cong sinh tự nhiên 1024-bit vừa xây dựng được.
4. **Về mặt công nghệ**: Toàn bộ hệ thống cơ sở được lập trình bằng ngôn ngữ Rust để đảm bảo hiệu năng tính toán cao ở mức ngôn ngữ hệ thống và sự an toàn bộ nhớ tuyệt đối từ cấp trình biên dịch.

[0] [1] E. Barker, "Recommendation for key management, part 1: general," National Institute of Standards; Technology, NIST Special Publication 800-57 Part 1 Revision 5, 2020. doi: 10.6028/NIST.SP.800-57pt1r5.

[0] [2] D. Shumow and N. Ferguson, "On the possibility of a back door in the NIST SP800-90 dual ec prng," in *CRYPTO 2007 rump session*, 2007.

[0] [3] CVE, "CVE-2014-0160: The heartbleed bug." `https://cve.mitre.org/cgi-bin/cvename.cgi?name=cve-2014-0160`, 2014.

[0] [4] M. Miller, “Trends, challenges, and strategic shifts in the software vulnerability mitigation landscape.” BlueHat IL, 2019.

CHƯƠNG 3
CƠ SỞ TOÁN HỌC

Chương này trình bày các khái niệm toán học nền tảng được sử dụng trong toàn bộ luận văn: cấu trúc trường hữu hạn, đường cong elliptic Short Weierstrass và các phép toán nhóm trên nó, cùng với các kỹ thuật số học tối ưu (Montgomery, Fermat) được cài đặt trực tiếp trong thư viện `curve1024`.

3.1. Định nghĩa Trường số nguyên tố

Trường số nguyên tố $\mathbb{F}_p$ là tập $\{0, 1, \ldots, p-1\}$ với phép cộng và nhân lấy modulo p, trong đó p là số nguyên tố. Đây là trường hữu hạn nhỏ nhất có đặc số p.

Các tính chất quan trọng:

- **Đóng kín**: kết quả của mọi phép toán đều thuộc $\mathbb{F}_p$.
- **Nghịch đảo nhân**: mọi phần tử $a \neq 0$ có nghịch đảo a^{-1} sao cho $a \cdot a^{-1} \equiv 1 \pmod{p}$.
- **Theo Định lý nhỏ Fermat**: $a^{p-1} \equiv 1 \pmod{p}$ với mọi $a \neq 0$, do đó $a^{-1} = a^{p-2} \pmod{p}$.

3.1.1. Cài đặt: `PrimeFieldConfig` và `PrimeFieldElement`

Trong thư viện `curve1024`, trường $\mathbb{F}_p$ được biểu diễn qua trait `PrimeFieldConfig` (mang các hằng số tĩnh) và struct `PrimeFieldElement<C>` (phần tử trường):

```
pub trait PrimeFieldConfig {
    const MODULUS: U1024;  // p: số nguyên tố trường
    const R2:      U1024;  // R² mod p, dùng cho Montgomery
    const N_PRIME: U1024;  // -p^{-1} mod 2^1024, dùng cho REDC
}

pub struct PrimeFieldElement<C: PrimeFieldConfig> {
    value: U1024,          // biểu diễn Montgomery: a.R mod p
}
```

Thiết kế này cho phép trình biên dịch mở inline toàn bộ hằng số tại thời gian biên dịch (zero-cost abstraction): hai trường khác nhau $\mathbb{F}_p$ và $\mathbb{F}_r$ dùng chung code nhưng có tham số riêng biệt không thể nhầm lẫn.

3.1.2. Các phép toán cơ bản

Cộng và trừ thực hiện bằng cộng/trừ số nguyên rồi hiệu chỉnh modulo:

```
// Cộng: (a + b) mod p
fn add(self, rhs: Self) -> Self {
    let (sum, carry) = self.value.carrying_add(&rhs.value);
    let (sub, borrow) = sum.borrowing_sub(&C::MODULUS);
    // Chọn kết quả không cần phép chia
    conditional_select(sum, sub, carry || !borrow)
}
```

Phép cộng không cần phép chia đầy đủ — chỉ cần một phép trừ có điều kiện. Chi phí: $O(n)$ với n là số limb (16 limb x 64 bit = 1024 bit).

Nhân dùng phép nhân Montgomery (xem §2.4).

Nghịch đảo dùng lũy thừa Fermat: $a^{-1} = a^{p-2} \pmod{p}$, thực hiện bởi hàm `pow` với phép nhân bình phương liên tiếp (square-and-multiply):

```
pub fn inv(&self) -> Self {
    self.pow(C::MODULUS - 2)   // a^(p-2) mod p
}
```

3.2. Đường cong elliptic Short Weierstrass

3.2.1. Định nghĩa

Trên $\mathbb{F}_p$ với $p > 3$, **đường cong elliptic dạng Short Weierstrass** là tập hợp:

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p \times \mathbb{F}_p \mid y^2 = x^3 + ax + b\} \cup \{\mathcal{O}\}$$

trong đó $\mathcal{O}$ là **điểm vô cực** (point at infinity) — phần tử đặc biệt không thuộc mặt phẳng affine, đóng vai trò phần tử đơn vị của nhóm.

Điều kiện không suy biến: Biệt thức $\Delta = -16(4a^3 + 27b^2) \neq 0$, đảm bảo đường cong không có điểm kỳ dị (cusp hay node). Kiểm tra này được thực hiện ngầm qua quá trình CM: với $D = -3$, luôn có $a = 0$ và điều kiện thu gọn thành $b \neq 0$.

3.2.2. Cấu trúc nhóm

Tập $E(\mathbb{F}_p)$ tạo thành một **nhóm Abel hữu hạn** với phép cộng điểm. Bậc của nhóm $\#E(\mathbb{F}_p) = p + 1 - t$ (định lý Hasse), trong đó $|t| \leq 2\sqrt{p}$ là vết Frobenius.

Trong cài đặt:

```
pub struct AffinePoint<C: SWCurveConfig> {
    pub x:           FieldElement<C::BaseField>,
    pub y:           FieldElement<C::BaseField>,
    pub is_infinite: bool,
}
```

3.3. Phép cộng điểm (Point Addition)

3.3.1. Quy tắc hình học

Phép cộng trên E được định nghĩa bằng quy tắc "dây-cung và tiếp tuyến":

- **Phần tử đơn vị:** $P + \mathcal{O} = \mathcal{O} + P = P$ với mọi P.
- **Phần tử nghịch đảo:** $-(x, y) = (x, -y)$, vì $P + (-P) = \mathcal{O}$.
- **Cộng hai điểm phân biệt** $P_1 \neq \pm P_2$: vẽ đường thẳng qua P_1, P_2, điểm giao thứ ba với đường cong là R', phản chiếu qua trục x cho $P_3 = P_1 + P_2 = -R'$.
- **Nhân đôi** $P + P = 2P$: dùng đường tiếp tuyến tại P.

3.3.2. Công thức đại số

Cho $P_1 = (x_1, y_1)$ và $P_2 = (x_2, y_2)$ trên $y^2 = x^3 + ax + b$, điểm tổng $P_3 = (x_3, y_3)$:

Cộng hai điểm phân biệt ($P_1 \neq P_2$):

$$\lambda = \frac{y_2 - y_1}{x_2 - x_1}, \quad x_3 = \lambda^2 - x_1 - x_2, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

Nhân đôi ($P_1 = P_2$):

$$\lambda = \frac{3x_1^2 + a}{2y_1}, \quad x_3 = \lambda^2 - 2x_1, \quad y_3 = \lambda(x_1 - x_3) - y_1$$

Mỗi công thức cần **1 phép nghịch đảo** (tính λ), 2–3 phép nhân, và 4–6 phép cộng/trừ trong $\mathbb{F}_p$.

3.3.3. Cài đặt

```
pub fn add(&self, rhs: &Self) -> Self {
    if self.is_infinite { return *rhs; }
    if rhs.is_infinite  { return *self; }
    if self.neg() == *rhs { return Self::infinite(); }
    if *self == *rhs  { return self.double(); }

    let lambda = (rhs.y - self.y) * (rhs.x - self.x).inv();
    let x3 = lambda.square() - self.x - rhs.x;
    let y3 = lambda * (self.x - x3) - self.y;
    Self::new(x3, y3)
}

pub fn double(&self) -> Self {
    if self.is_infinite || self.y.is_zero() { return Self::infinite(); }
    let three = FieldElement::new(U1024::from(3));
    let two   = FieldElement::new(U1024::from(2));
    let lambda = (three * self.x.square() + C::COEFF_A) * (two * self.y).inv()
    let x3 = lambda.square() - self.x - self.x;
    let y3 = lambda * (self.x - x3) - self.y;
    Self::new(x3, y3)
}
```

Hàm `new` gọi `assert!(point.is_on_curve())` để đảm bảo mọi điểm kết quả đều hợp lệ — bắt lỗi nhanh trong quá trình phát triển.

3.4. Phép nhân vô hướng (Scalar Multiplication)

3.4.1. Thuật toán Double-and-Add

Phép nhân vô hướng $[k]P = P + P + \cdots + P$ (k lần) được tính hiệu quả bằng **Double-and-Add** (tương tự binary exponentiation), với độ phức tạp $O(\log k)$ thay vì $O(k)$:

```
Input: P in E(F_p), k in [0, r-1]
Output: [k]P

R ← O (điểm vô cực)
B ← P
for i = 0 to 1023:
```

```
    if bit i của k bằng 1:
        R ← R + B
    B ← 2B
return R
```

Với $k \approx 2^{512}$ (1024 bit), thuật toán thực hiện tối đa 1024 lần nhân đôi và trung bình 512 lần cộng điểm.

3.4.2. Cài đặt

```rust
pub fn mul(&self, scalar: &U1024) -> Self {
    let mut result = Self::infinite();
    let mut base   = *self;
    for i in 0..1024 {
        let limb_idx = i / 64;
        let bit_idx  = i % 64;
        if (scalar.0[limb_idx] >> bit_idx) & 1 == 1 {
            result = result.add(&base);
        }
        base = base.double();
    }
    result
}
```

Mỗi lần gọi `mul` thực hiện đúng 1024 lần `double` và tối đa 1024 lần `add`. Chi phí tính toán chủ yếu đến từ phép nghịch đảo trong $\mathbb{F}_p$ (mỗi lần `add`/`double` cần 1 nghịch đảo, tức p^{p-2} — một lũy thừa 1024-bit).

3.4.3. Tầm quan trọng

Phép nhân vô hướng là hàm cốt lõi của mọi sơ đồ mật mã ECC:

Thao tác	Ký hiệu	Ứng dụng
Sinh khóa công khai	$Q = [d]G$	Từ khóa riêng d
Cam kết ký (Schnorr/ECDSA)	$R = [k]G$	Nonce commitment
Xác minh Schnorr	$[s]G$ và $[e]Q$	Hai lần nhân vô hướng
Xác minh ECDSA	$[u_1]G + [u_2]Q$	Hai lần nhân vô hướng

3.5. Tối ưu số học: Phép nhân Montgomery

3.5.1. Vấn đề

Phép nhân thông thường $a \cdot b \pmod p$ đòi hỏi một phép chia đầy đủ để lấy số dư — rất đắt với số 1024-bit. Montgomery đề xuất thực hiện phép nhân trong **không gian Montgomery** để tránh phép chia.

3.5.2. Biến đổi Montgomery

Chọn $R = 2^{1024}$ (cơ số nguyên tố cùng nhau với p). **Dạng Montgomery** của a là $\hat{a} = a \cdot R \pmod p$.

Phép nhân Montgomery (**REDC**) tính $\hat{a} \cdot \hat{b} \cdot R^{-1} \pmod p$ từ $\hat{a}$ và $\hat{b}$, chỉ dùng phép chia cho R (shift phải 1024 bit — rất rẻ) thay vì chia cho p.

Phép tính	Thông thường	Montgomery
$a \cdot b \pmod p$	cần chia cho p	shift + trừ có điều kiện
Chi phí	$O(n^2)$ + chia	$O(n^2)$ không chia

Quy trình REDC (với ba hằng số biên dịch: $R^2 \bmod p$, $-p^{-1} \bmod R$): 1. Nhân $\text{lo}, \text{hi} = \hat{a} \cdot \hat{b}$ 2. $m = \text{lo} \cdot N' \bmod R$ với $N' = -p^{-1} \bmod R$ 3. $t = (\text{hi}, \text{lo}) + m \cdot p$ (chỉ giữ phần trên R) 4. Nếu $t \geq p$: $t \leftarrow t - p$

```
fn reduce(lo: &U1024, hi: &U1024) -> U1024 {
    let (m, _)        = lo.widening_mul(&C::N_PRIME);    // m = lo . N' mod R
    let (mn_lo, mn_hi) = m.widening_mul(&C::MODULUS);   // m . p
    let (_, carry_lo)  = lo.carrying_add(&mn_lo);
    let (t, carry_hi)  = hi.carrying_add(&mn_hi);
    // Bước hiệu chỉnh cuối: trừ p nếu cần
    let (sub, borrow) = t.borrowing_sub(&C::MODULUS);
    if carry_hi || carry_lo { sub } else if !borrow { sub } else { t }
}
```

3.5.3. Chuyển đổi vào/ra không gian Montgomery

- **Chuyển vào:** `PrimeFieldElement::new(a)` tính $a \cdot R^2 \cdot R^{-1} = a \cdot R \pmod p$.
- **Chuyển ra:** `to_u1024()` gọi `reduce(value, 0)` = $\hat{a} \cdot R^{-1} = a \pmod p$.

Trong thực tế, hầu hết tính toán diễn ra trong không gian Montgomery — chỉ chuyển ra khi cần so sánh hay xuất kết quả, giúp loại bỏ hoàn toàn phép chia trong vòng lặp tính toán chính.

3.6. Lũy thừa nhanh (Square-and-Multiply)

Hàm `pow` thực hiện lũy thừa $a^e \pmod p$ bằng phương pháp **square-and-multiply** với bảo vệ timing side-channel qua `conditional_select`:

```
pub fn pow(&self, exp: U1024) -> Self {
    let mut res  = Self::one();
    let mut base = *self;
    for i in 0..16 {            // 16 limb x 64 bit = 1024 bit
        let mut limb = exp.0[i];
        for _ in 0..64 {
            let bit     = (limb & 1) as u8;
            let product = res * base;
            // Chọn res hoặc product dựa trên bit, không dùng nhánh if
            res  = Self::conditional_select(&res, &product, bit.into());
            base = base.square();
            limb >>= 1;
        }
    }
    res
}
```

`conditional_select` từ crate `subtle` đảm bảo cả hai nhánh thực hiện cùng số lệnh, loại bỏ thông tin qua thời gian thực thi (constant-time execution).

Nghịch đảo $a^{-1} = a^{p-2}$ dùng `pow` với $e = p - 2$: 1024 lần bình phương và ~512 lần nhân (trung bình), tổng ≈ 1536 phép nhân Montgomery.

CHƯƠNG 4
PHƯƠNG PHÁP XÂY DỰNG ĐƯỜNG CONG ELLIPTIC

Chương này trình bày trọng tâm lý thuyết của luận văn: các phương pháp xây dựng đường cong elliptic có bậc nhúng (embedding degree) xác định, phục vụ cho các ứng dụng mật mã dựa trên phép ghép cặp (pairing-based cryptography). Chúng ta sẽ đi từ nền tảng lý thuyết của Phương pháp Nhân phức (CM) và Đa thức lớp Hilbert, đến thuật toán Cocks-Pinch truyền thống, và cuối cùng là thuật toán Cocks-Pinch Cải tiến được đề xuất trong luận văn này, với khả năng sinh tham số trường cơ sở có cấu trúc NTT-friendly nhằm kháng lại tấn công rây trường số tháp (Tower Number Field Sieve - TNFS).

4.1. Ý tưởng cốt lõi

Phương pháp Nhân phức (CM) là công cụ nền tảng để **xây dựng đường cong elliptic có số điểm định trước**. Thay vì chọn ngẫu nhiên các hệ số a, b rồi đếm điểm, CM đi ngược lại: bắt đầu từ bậc nhóm mong muốn $\#E(\mathbb{F}_p) = p + 1 - t$ rồi suy ngược ra phương trình đường cong.

Ý tưởng xuất phát từ lý thuyết số học phức: mỗi đường cong elliptic E sở hữu một **vành nội cấu** (endomorphism ring) $\mathrm{End}(E)$. Với đường cong thông thường (không siêu dị), vành này đẳng cấu với một **thứ tự trong trường số** $\mathbb{Q}(\sqrt{D})$, trong đó $D < 0$ là số nguyên âm gọi là **biệt thức CM**. Giá trị D đặc trưng hoàn toàn cho "loại" đường cong về mặt số học phức.

4.1.1. Điều kiện CM và phương trình Diophantine

Để tồn tại một đường cong elliptic trên $\mathbb{F}_p$ có vết Frobenius t và biệt thức CM là D, điều kiện cần và đủ là phương trình Diophantine sau phải có nghiệm nguyên:

$$4p = t^2 + |D| \cdot v^2$$

trong đó p là đặc số trường, t là vết Frobenius, v là một số nguyên dương, và $D < 0$. Phương trình này **ràng buộc trực tiếp** bộ tham số (p, t, D) với nhau; không phải mọi bộ ba nào cũng có thể thỏa mãn đồng thời.

Ví dụ: Với $D = -3$ (biệt thức của secp256k1, BLS12-381), phương trình trở thành

$4p = t^2 + 3v^2$.

4.1.2. Đa thức lớp Hilbert $H_D(x)$

Khi đã có D, bước tiếp theo là tính **Đa thức lớp Hilbert** (Hilbert class polynomial) $H_D(x)$. Đây là đa thức hệ số nguyên được xây dựng từ lý thuyết số học phức mà nghiệm phức của nó chính là các j**-invariant** của tất cả đường cong elliptic phức $\mathbb{C}/\mathcal{O}$ với thứ tự CM là $\mathcal{O}$.

Bậc của $H_D(x)$ bằng **số lớp** (class number) $h(D)$ — một bất biến số học của thứ tự $\mathbb{Z}[\sqrt{D}]$. Với $|D|$ nhỏ:

$	D	$	$h(D)$	$H_D(x)$
3	1	x		
4	1	$x - 1728$		
7	1	$x + 3375$		
11	1	$x + 32768$		
23	3	$x^3 + \cdots$		

Với $|D| = 3$ và $|D| = 4$, đa thức này bậc 1, nên j**-invariant được xác định ngay lập tức**: $j = 0$ và $j = 1728$ tương ứng. Đây là lý do tại sao secp256k1 ($j = 0$, $D = -3$) và nhiều đường cong BLS có dạng đơn giản $y^2 = x^3 + b$.

4.1.3. Quy trình CM đầy đủ

Cho bộ tham số (p, t, D) thỏa mãn $4p = t^2 + |D|v^2$:

1. **Tính** j**-invariant**: Tìm nghiệm $j_0 \in \mathbb{F}_p$ của $H_D(x) \pmod p$.
2. **Suy ra hệ số đường cong**: Từ j_0, tính $c = j_0 \cdot (1728 - j_0)^{-1} \pmod p$, sau đó:
   - $a = 3c$, $b = 2c$ (trường hợp tổng quát $j_0 \neq 0, 1728$)
   - $a = 0$, b tùy ý (khi $j_0 = 0$, tức $D = -3$)
   - a tùy ý, $b = 0$ (khi $j_0 = 1728$, tức $D = -4$)
3. **Kiểm tra xoắn (Twist test)**: Đường cong $E : y^2 = x^3 + ax + b$ và đường cong xoắn $E' : y^2 = x^3 + a\delta^2 x + b\delta^3$ (với δ là phần tử không phải thặng dư bậc hai) có tổng số điểm $\#E + \#E' = 2(p + 1)$. Ta cần chọn đúng đường cong có $r \mid \#E$.
4. **Tìm điểm sinh (Generator)**: Nhân ngẫu nhiên một điểm trên đường cong với hệ số cofactor $h = \#E/r$ để tìm điểm sinh G có bậc chính xác là r.

4.1.4. *Trường hợp $D = -3$: Xoắn sextic*

Với $D = -3$, tình huống phức tạp hơn vì đường cong $y^2 = x^3 + b$ có **6 xoắn** (sextic twists) thay vì chỉ 2. Sáu vết Frobenius khả dĩ là:

$$t, \quad -t, \quad \frac{t \pm 3v}{2}, \quad \frac{-t \pm 3v}{2}$$

trong đó $4p = t^2 + 3v^2$. Để xác định xoắn đúng, ta lần lượt thử các giá trị b nhỏ và kiểm tra số điểm thực sự của đường cong $y^2 = x^3 + b$ bằng cách tính phép nhân vô hướng $[p + 1 - t']P$ với điểm ngẫu nhiên P.

Trong cài đặt của luận văn (tệp `complex_multiplication.rs`), hàm `find_twist` thực hiện:

```
// Sáu bậc nhóm khả dĩ cho D = -3 (j = 0)
let candidates = [
    p1.borrowing_sub(&config.t).0,     //  p + 1 - t
    p1.carrying_add(&config.t).0,      //  p + 1 + t
    p1.borrowing_sub(&t3p).0,          //  p + 1 - (t+3v)/2
    p1.carrying_add(&t3p).0,           //  p + 1 + (t+3v)/2
    p1.borrowing_sub(&t3m).0,          //  p + 1 - (t-3v)/2
    p1.carrying_add(&t3m).0,           //  p + 1 + (t-3v)/2
];
// Tìm bậc nào chia hết cho r, rồi tìm b tương ứng
```

4.2. Tham số đầu vào của hệ thống

Trước khi đi vào chi tiết thuật toán, phần này giải thích ý nghĩa và cơ sở lựa chọn của 7 tham số đầu vào trong cài đặt thực tế (`cocks_pinch.rs`):

```
let k                    = 18u64;
let d                    = U1024::from(3);
let target_r_bits        = 512;
let target_p_bits        = 1024;
let max_attempts         = 100_000u64;
let min_scalar_two_adicity = 32u32;
let min_base_two_adicity   = 32u32;
```

4.2.1. `k = 18` — *Bậc nhúng (Embedding Degree)*

Bậc nhúng k xác định **trường mở rộng** $\mathbb{F}_{p^k}$ nơi phép ghép cặp Weil/Tate được tính toán. Giá trị $k = 18$ được chọn vì:

- **Cân bằng bảo mật:** Với $r \approx 2^{512}$ và $p \approx 2^{1024}$, kích thước trường mở rộng là $p^k \approx 2^{1024\times 18} = 2^{18432}$ bit. Theo tiêu chuẩn NFS hiện tại, bài toán DLP trên $\mathbb{F}_{p^{18}}$ ở mức bảo mật ≥ 256 bit — tương đương với mức bảo mật của khóa $r \approx 2^{512}$.
- **Hỗ trợ Đa thức Cyclotomic bậc 18:** $\Phi_{18}(T) = T^6 - T^3 + 1$ cho phép sinh r hiệu quả, có cấu trúc NTT-friendly tự nhiên khi $T \equiv 0 \pmod{2^k}$ (xem §3.3.1).
- **Thực tế:** $k = 18$ là bậc nhúng phổ biến trong họ đường cong KSS18, phù hợp với mức bảo mật hậu lượng tử cao hơn các đường cong BLS12 (dùng trong Ethereum 2.0).

4.2.2. `d = 3` — *Biệt thức CM ($|D|$)*

Biệt thức CM $D = -3$ được chọn vì những ưu điểm nổi bật:

- **Đa thức Hilbert bậc 1:** $H_{-3}(x) = x$, nên j-invariant bằng 0 và phương trình đường cong có dạng cực kỳ đơn giản $y^2 = x^3 + b$. Không cần giải đa thức bậc cao.
- **Phổ biến trong thực tế:** secp256k1 (Bitcoin), BLS12-381 (Ethereum), và nhiều đường cong quan trọng khác đều dùng $D = -3$.
- **Điều kiện CM đơn giản:** Phương trình $4p = t^2 + 3v^2$ dễ giải hơn với hầu hết các giá trị (t, v) nguyên, tăng xác suất tìm được p nguyên tố trong lưới nâng.

4.2.3. `target_r_bits = 512` — *Kích thước bậc nhóm r*

Tham số này xác định **mức bảo mật** của bài toán logarit rời rạc trên đường cong (ECDLP):

- Với $r \approx 2^{512}$, thuật toán Pollard-rho tốt nhất cần $\approx 2^{256}$ phép tính, đạt mức bảo mật **256-bit** — vượt tiêu chuẩn NIST SP 800-57 cho ứng dụng đến năm 2030+ và phù hợp với kịch bản hậu lượng tử (post-quantum).
- Đây cũng là kích thước của **khóa riêng** (private key scalar) và **kích thước chữ ký** trong các giao thức Schnorr và ECDSA trên đường cong này.

4.2.4. `target_p_bits = 1024` — *Kích thước trường cơ sở* p

Kích thước trường cơ sở tác động đến hai yếu tố:

- **Bảo mật DLP trên $\mathbb{F}_{p^k}$:** Với $p \approx 2^{1024}$ và $k = 18$, kích thước trường mở rộng là 2^{18432} bit. Theo ước lượng NFS/TNFS, điều này cho mức bảo mật ≥ 250 bit — phù hợp với ứng dụng yêu cầu bảo mật dài hạn.
- **Hiệu năng:** Kích thước 1024 bit làm cho mỗi phép nhân trường tốn gấp đôi so với 512 bit về mặt thời gian CPU, nhưng vẫn trong giới hạn thực tế với phép nhân Montgomery đa độ chính xác.
- **Tỉ số ρ:** $\rho = \log_2 p / \log_2 r = 1024/512 = 2$. Đây là tỉ số điển hình của Cocks-Pinch (so với $\rho = 1.5$ của BLS12 hay $\rho = 1$ của BN).

4.2.5. `max_attempts = 100_000` — *Số lần thử tối đa*

Giới hạn vòng lặp tìm kiếm an toàn. Trong thực nghiệm, thuật toán CP cải tiến thường tìm được kết quả trong **100–6000 lần thử** (tùy cấu hình NTT), do đó `100_000` là giới hạn rất thoải mái đảm bảo không bao giờ bị timeout trong điều kiện bình thường. Nếu vượt giới hạn này (không bao giờ xảy ra trong thực tế), hệ thống báo lỗi thay vì chạy mãi mãi.

4.2.6. `min_scalar_two_adicity = 32` — *Two-adicity tối thiểu của* $r - 1$

Tham số này ràng buộc **cấu trúc NTT-friendly của trường vô hướng** $\mathbb{F}_r$:

$$2^{32} \mid (r-1) \quad \Longleftrightarrow \quad r = d \cdot 2^{32} + 1 \text{ với } d \text{ lẻ}$$

Ý nghĩa thực tế: trường $\mathbb{F}_r$ chứa một căn nguyên thủy 2^{32}-th của đơn vị, cho phép thực hiện NTT trên các đa thức bậc lên đến $2^{32} \approx 4 \times 10^9$. Đây là yêu cầu cốt lõi để tạo chứng minh ZK hiệu quả:

- Trong Groth16/PLONK, phép nhân đa thức $A(x) \cdot B(x)$ với mạch có 2^{20} ràng buộc cần NTT bậc $\geq 2^{21}$ — thỏa mãn với 2^{32}.
- BLS12-381 (chuẩn công nghiệp) có r two-adicity = 32; hệ thống này đạt ≥ 33, **vượt chuẩn BLS12-381**.

Giá trị 32 đạt được **không cần rejection sampling** nhờ ràng buộc $T \equiv 0 \pmod{2^{11}}$ (vì $v_2(r-1) = 3 \cdot v_2(T) \geq 3 \times 11 = 33 \geq 32$).

4.2.7. `min_base_two_adicity = 32` — Two-adicity tối thiểu của $p-1$

Tương tự, tham số này ràng buộc **cấu trúc NTT-friendly của trường cơ sở** $\mathbb{F}_p$:

$$2^{32} \mid (p-1) \quad \Longleftrightarrow \quad p = d \cdot 2^{32} + 1$$

Đây cũng là yêu cầu của thuật toán Miller trong phép ghép cặp: các biểu thức trung gian (line evaluations) được tính trong $\mathbb{F}_{p^k}$ và có thể được tăng tốc bằng NTT nếu p có cấu trúc NTT. Ngoài ra:

- $p = d \cdot 2^{32} + 1$ kháng TNFS tốt hơn p tùy ý (xem §3.3), do cấu trúc $p-1$ không có nhân tử nhỏ ngẫu nhiên mà thuật toán rây có thể khai thác.
- Giá trị này đạt được bằng cách mở rộng lưới nâng (h_t, h_y) trong hàm `try_lift_to_prime`.

4.2.8. Tổng hợp: Đặc trưng đường cong mục tiêu

Từ 7 tham số trên, đường cong được xây dựng có đặc trưng:

Thuộc tính	Giá trị	Ý nghĩa
Phương trình	$y^2 = x^3 + b$	$D = -3$, $j = 0$, dạng tối giản
Bậc nhúng	$k = 18$	Phép ghép cặp trong $\mathbb{F}_{p^{18}}$
Kích thước r	512 bit	Bảo mật ECDLP 256-bit
Kích thước p	1024 bit	Bảo mật DLP ≥ 250-bit
Two-adicity r	≥ 33	NTT/FFT trên $\mathbb{F}_r$ đến bậc 2^{33}
Two-adicity p	≥ 32	NTT/FFT trên $\mathbb{F}_p$ đến bậc 2^{32}
Dạng r	$d \cdot 2^{33} + 1$	NTT-friendly scalar field
Dạng p	$d \cdot 2^{34} + 1$	NTT-friendly base field (kháng TNFS)

4.3. Thuật toán Cocks-Pinch truyền thống

4.3.1. Vấn đề: Xây dựng ngược từ r

Phương pháp CM truyền thống xuất phát từ p được chọn trước, đòi hỏi giải phương trình $4p = t^2 + |D|v^2$ — một bài toán biểu diễn số nguyên bởi dạng toàn phương (quadratic form) không hề dễ trong không gian lớn.

Thuật toán Cocks-Pinch (CP) đảo chiều bài toán: **bắt đầu từ** r **rồi xây dựng** p. Đây là cách tiếp cận tự nhiên hơn khi mục tiêu là đảm bảo r có kích thước an toàn (ví dụ: 512 bit) với bậc nhúng k cho trước.

4.3.2. Điều kiện bậc nhúng

Bậc nhúng k của đường cong là **số nguyên dương nhỏ nhất** sao cho $r \mid p^k - 1$, hay tương đương $p \equiv 1 \pmod{r}$ khi $k = 1$, hoặc tổng quát hơn p là nghiệm của $\Phi_k(x) \equiv 0 \pmod{r}$.

Điều kiện cụ thể: $r \mid \Phi_k(p)$, trong đó Φ_k là đa thức cyclotomic. Điều này có nghĩa là tồn tại u nguyên tố cùng nhau với k sao cho:

$$p \equiv \rho^i \pmod{r} \quad \text{với } \rho \text{ là nghiệm nguyên thủy của } \Phi_k(x) \equiv 0 \pmod{r}$$

4.3.3. Thuật toán CP cho $k = 18$

Với $k = 18$ và $D = -3$, bậc nhóm $r = \Phi_{18}(T) = T^6 - T^3 + 1$ với T nguyên tùy ý. Các bước:

Bước 1 — Sinh r **từ** T**:**

$$r = \Phi_{18}(T) = T^6 - T^3 + 1$$

Kiểm tra tính nguyên tố của r bằng thuật toán Miller-Rabin.

Bước 2 — Tính $\sqrt{-D} \pmod{r}$**:**

Cần $\beta = \sqrt{-3} \pmod{r}$. Do $r \equiv 1 \pmod{3}$ (do cấu trúc của Φ_{18}), phần tử này luôn tồn tại. Dùng thuật toán Tonelli–Shanks.

Bước 3 — Tính t_0, y_0 **modulo** r**:**

Với mỗi i thỏa $\gcd(i, k) = 1$ và $1 \leq i < k$:

$$t_0 = T^i + 1 \pmod{r}, \quad y_0 = \frac{t_0 - 2}{\beta} \pmod{r}$$

Bước 4 — Nâng (lift) lên số nguyên:

Tìm $t, y \in \mathbb{Z}$ với $t \equiv t_0 \pmod{r}$, $y \equiv y_0 \pmod{r}$ sao cho:

$$p = \frac{t^2 + 3y^2}{4}$$

là số nguyên và là số nguyên tố. Để làm điều này, thử các dịch chuyển nhỏ:

$$t = t_0 + h_t \cdot r, \quad y = y_0 + h_y \cdot r, \quad h_t, h_y \in \{-20, \ldots, 20\}$$

cho đến khi $\frac{t^2+3y^2}{4}$ là số nguyên tố đúng kích thước mục tiêu.

Trong cài đặt (`cocks_pinch.rs`), hàm `try_lift_to_prime` thực hiện tìm kiếm này:

```
for ht in -half_range..=half_range {
    for hy in -half_range..=half_range {
        let t = apply_lift(lp.t0, r, ht as i32);
        let y = apply_lift(lp.y0, r, hy as i32);
        // p = (t² + 3y²) / 4
        let numerator = t² + 3y²;
        if numerator % 4 != 0 { continue; }
        let p = numerator / 4;
        if is_prime(&p) { return Some(CurveParams { p, r, t, y, .. }); }
    }
}
```

4.3.4. Phân tích độ phức tạp trung bình

Mật độ số nguyên tố lân cận N là khoảng $1/\ln N$, tức $1/1024 \ln 2 \approx 1/709$. Với lưới $(h_t, h_y) \in [-20, 20]^2$ tạo ra $41 \times 41 = 1681$ ứng viên p mỗi lần, tỉ lệ thành công mỗi lần sinh r là khoảng $1681/709 \approx 2.4$ ứng viên p nguyên tố. Trong thực tế, thường cần khoảng **100–500 lần sinh** r để tìm được bộ tham số hợp lệ.

4.4. Thuật toán Cocks-Pinch Cải tiến

4.4.1. Động lực: Tấn công rây trường số tháp (TNFS)

Bảo mật của phép ghép cặp phụ thuộc vào độ khó của **bài toán logarit rời rạc** (DLP) trên trường mở rộng $\mathbb{F}_{p^k}$. Năm 2016, Barbulescu và Duquesne chỉ ra rằng thuật toán **Tower Number Field Sieve (TNFS)** — một biến thể của NFS — có thể giảm đáng kể độ phức tạp tấn công DLP trên $\mathbb{F}_{p^k}$ khi k composite và p có cấu trúc đặc biệt. Điều này **phá vỡ ước lượng bảo mật ban đầu** của các đường cong như BN256 (từ ~128 bit xuống còn ~100 bit hiệu quả).

Để kháng lại TNFS, cần đảm bảo trường cơ sở của đường cong có **cấu trúc tốt** ở cả hai trường: trường vô hướng $\mathbb{F}_r$ (phục vụ tính toán ZK proof) và trường cơ sở $\mathbb{F}_p$ (phục vụ phép ghép cặp).

4.4.2. Yêu cầu NTT-friendly

Một số nguyên tố q được gọi là **NTT-friendly** (hoặc FFT-friendly) nếu $q-1$ chia hết cho một lũy thừa đủ lớn của 2, tức là:

$$2^s \mid (q-1) \quad \text{với } s \text{ đủ lớn}$$

Số s được gọi là **số hạng hai-adicity** (two-adicity) của q. Ý nghĩa: trường $\mathbb{F}_q$ chứa một căn nguyên thủy 2^s-th của đơn vị, cho phép áp dụng **Biến đổi Lý thuyết số (NTT)** — tức FFT trên trường hữu hạn — để nhân đa thức bậc cao trong $O(n \log n)$ thay vì $O(n^2)$.

Đây là yếu tố *quyết định* hiệu năng trong: - Sinh chứng minh ZK (Groth16, PLONK): tính $H(x) = (A(x) \cdot B(x) - C(x))/Z(x)$ - Các cam kết đa thức (KZG, FRI) - Thuật toán MSM (Multi-scalar multiplication) biến thể NTT

Tương quan với TNFS: Một trường $\mathbb{F}_p$ với two-adicity cao có $p-1 = d \cdot 2^s$, nghĩa là $p = d \cdot 2^s + 1$. Cấu trúc này khiến cho tháp trường (tower field extension) $\mathbb{F}_{p^k}$ có đặc điểm tốt hơn từ góc độ kháng TNFS: các thuật toán rây cần khai thác cấu trúc phân tích của $p-1$ để tìm quan hệ, do đó khi $p-1$ có cấu trúc **không phân tích tùy tiện** mà theo dạng chuẩn $d \cdot 2^s + 1$, sự khai thác thông tin này trở nên khó hơn.

4.4.3. Cải tiến 1: NTT-friendly r qua ràng buộc T

Nhận xét then chốt: Vì $r = \Phi_{18}(T) = T^6 - T^3 + 1$, ta có:

$$r - 1 = T^3(T^3 - 1)$$

Nếu $T \equiv 0 \pmod{2^k}$, thì $T^3 \equiv 0 \pmod{2^{3k}}$, và do đó:

$$v_2(r-1) = v_2(T^3(T^3-1)) = 3k$$

(vì $T^3 \equiv 0 \pmod{2^{3k}}$ và $T^3 - 1 \equiv -1 \pmod 2$ là lẻ).

Kết luận: Để đảm bảo two-adicity$(r-1) \geq s$, chỉ cần **chọn** $T \equiv 0 \pmod{2^{\lceil s/3 \rceil}}$. Đây là ràng buộc *không cần rejection sampling* — mọi T thỏa mãn điều kiện này đều cho r với two-adicity ít nhất s.

Trong cài đặt:

```
// Chọn T là bội số của 2^ceil(s/3): đảm bảo two_adicity(r-1) >= s
let t_align = min_scalar_two_adicity.div_ceil(3);
let step = U1024::ONE.shl(t_align as usize);
// t_val luôn là bội số của step
let t_val = t_base + U1024::rand(&t_steps) * step;
let r = cyclotomic_phi18(&t_val);
// Đảm bảo r thỏa mãn: two_adicity(r-1) = 3 * v_2(T) >= s
```

Với `min_scalar_two_adicity = 32`, ta cần $\lceil 32/3 \rceil = 11$, tức $T \equiv 0 \pmod{2^{11}}$. Điều này **không làm giảm không gian tham số** đáng kể — vẫn còn $2^{85}/2^{11} = 2^{74}$ ứng viên T trong dải hợp lệ.

4.4.4. Cải tiến 2: NTT-friendly p qua mở rộng lưới nâng

Sau khi có $t_0, y_0 \pmod{r}$, tham số p được xác định bởi:

$$p = \frac{(t_0 + h_t r)^2 + 3(y_0 + h_y r)^2}{4}$$

Vì $r \equiv 0 \pmod{2^{3k}}$ (do $T \equiv 0 \pmod{2^k}$), các bit thấp hơn $3k$ của $p - 1$ chỉ phụ thuộc vào t_0, y_0 (cố định cho mỗi r). Tuy nhiên, các bit từ vị trí $3k$ trở lên bị ảnh hưởng bởi h_t, h_y.

Để đạt two-adicity$(p - 1) \geq s_p > 3k$, ta cần mở rộng lưới (h_t, h_y) ra ngoài phạm vi ± 20 mặc định:

$$\text{extra} = \max(0,\ s_p - 3k), \quad \text{half_range} = 20 + 2^{\text{extra}}$$

Điều này đảm bảo có đủ ứng viên h_t, h_y để **bao phủ mọi tổ hợp bit từ vị trí** $3k$ **đến** s_p trong $p - 1$.

```
struct LiftParams<'a> {
    t0: &'a U1024,
    y0: &'a U1024,
    target_p_bits: usize,
    min_base_two_adicity: u32,   // s_p
    r_two_adicity: u32,          // 3k
}
```

```
// Mở rộng lưới tìm kiếm tỉ lệ với khoảng cách giữa s_p và 3k
let extra = lp.min_base_two_adicity
    .saturating_sub(lp.r_two_adicity)
    .min(10); // giới hạn để lưới không bùng nổ
let half_range = 20i64 + (1i64 << extra);
```

4.4.5. Phân tích hiệu năng của thuật toán cải tiến

Bảng dưới so sánh thuật toán CP gốc và CP Cải tiến với cùng mục tiêu ($r \approx 2^{512}, p \approx 2^{1024}, k = 18$):

Tham số	CP Truyền thống	CP Cải tiến ($s_r = 32, s_p = 32$)
Ràng buộc trên T	Không có	$T \equiv 0 \pmod{2^{11}}$
two-adicity của $r - 1$	Ngẫu nhiên (~1-3)	≥ 33 (đảm bảo)
two-adicity của $p - 1$	Ngẫu nhiên (~1)	≥ 32 (với lưới mở rộng)
Lưới (h_t, h_y)	$[-20, 20]^2$	$[-21, 21]^2$ (khi $s_p \leq 3k$)
Chi phí mỗi lần thử	~50ms	~50ms
Số lần thử trung bình	~100–500	~3.000–6.000
Thời gian thực tế	~2–10 giây	~30–60 giây
r hỗ trợ NTT?	Không	Có (bậc 2^{33})
p hỗ trợ NTT?	Không	Có (bậc 2^{34})

Kết quả thực nghiệm điển hình:

```
r two-adicity: 2^33 | (r-1)  →  r = d.2^33 + 1, NTT đến bậc 2^33
p two-adicity: 2^34 | (p-1)  →  p = d.2^34 + 1, NTT đến bậc 2^34
Thời gian tổng: 32.81s
```

4.4.6. So sánh với BLS12-381

Đường cong BLS12-381 — chuẩn công nghiệp hiện tại trong ZK proof — có: - r two-adicity = **32** (đủ cho ZK circuits đến 2^{32} ràng buộc) - p two-adicity = **32**

Đường cong được xây dựng trong luận văn này đạt **33–34**, tức là **vượt BLS12-381 về cấu trúc NTT** trong khi kích thước khóa lớn hơn ($p \approx 2^{1024}$ so với $p \approx 2^{381}$), cung cấp mức bảo mật hậu lượng tử cao hơn trong ngữ cảnh phép ghép cặp.

4.4.7. Giới hạn và hướng mở rộng

Thuật toán cải tiến vẫn có giới hạn thực tế:

- **Two-adicity tối đa đạt được**: Khi $s_p > 3k + 10$, lưới (h_t, h_y) phải có half_range $> 2^{10} = 1024$, số lần lặp vượt 10^6 mỗi ứng viên r, khiến tìm kiếm không thực tế. Nếu cần s_p cực cao (>50), cần dùng họ đường cong chuyên dụng thiết kế từ đầu như Pasta curves (Pallas/Vesta).
- **Tỉ số** ρ: Thuật toán CP vốn cho tỉ số $\rho = \log p / \log r \approx 2$, không tối ưu bằng các họ đường cong chuyên biệt như BN256 ($\rho = 1$). Đây là sự đánh đổi cố hữu của phương pháp CP.
- **Chỉ** $k = 18$ **và** $D = -3$ được cài đặt trong hệ thống hiện tại. Mở rộng sang $k = 12, 24$ đòi hỏi thay đổi đa thức cyclotomic và công thức CM.

CHƯƠNG 5
SƠ ĐỒ CHỮ KÝ SỐ

Chương này trình bày ba sơ đồ chữ ký số được xây dựng trên nền tảng đường cong elliptic đã sinh ở Chương 3: **Schnorr**, **ECDSA**, và **chữ ký tổng hợp BLS**. Với mỗi sơ đồ, chúng ta đi qua lý thuyết cốt lõi, phân tích cấu trúc dữ liệu và cài đặt thực tế bằng Rust. Kết quả đo hiệu năng và phân tích bảo mật được trình bày riêng tại **Chương 5**.

5.1. Cặp khóa (KeyPair)

Mọi sơ đồ chữ ký đều dùng chung cấu trúc cặp khóa:

```rust
pub struct KeyPair<C: SWCurveConfig> {
    pub private_key: U1024,        // d in [1, r-1], bí mật
    pub public_key:  AffinePoint<C>, // Q = [d]G, công khai
}
```

- **Khóa riêng** `d`: một số nguyên ngẫu nhiên trong $[1, r-1]$, sinh bởi CSPRNG (`U1024::rand`).
- **Khóa công khai** `Q = [d]G`: tính bằng phép nhân vô hướng. Với $r \approx 2^{512}$, tính Q từ d là tính khả thi (mili giây), nhưng bài toán ngược (tính d từ Q) là ECDLP — độ phức tạp $O(2^{256})$.

Cặp khóa được lưu/đọc từ file nhị phân 128 byte (big-endian) với quyền `0600` trên Unix:

```rust
impl<C: SWCurveConfig> KeyPair<C> {
    pub fn generate() -> Self {
        let private_key = U1024::rand(&C::ORDER);
        let public_key  = C::generator().mul(&private_key);
        Self { private_key, public_key }
    }
    pub fn save(&self, path: &str) -> io::Result<()> {
        fs::write(path, self.private_key.to_be_bytes())?;
```

```
        fs::set_permissions(path, Permissions::from_mode(0o600))?;
        Ok(())
    }
}
```

5.1.1. Hàm băm mở rộng

Vì $r \approx 2^{512}$ và SHA-256 chỉ cho 256 bit, hệ thống dùng **4 lần SHA-256** với tiền tố khác nhau để sinh 1024 bit, rồi cắt lấy 512 bit thấp khi cần:

```
pub fn hash_message(message: &[u8]) -> U1024 {
    let mut buf = [0u8; 128];
    for i in 0..4 {
        let mut h = Sha256::new();
        h.update([i as u8]);
        h.update(message);
        buf[i*32..(i+1)*32].copy_from_slice(&h.finalize());
    }
    U1024::from_be_bytes(&buf)
}
```

Cấu trúc này tương đương với **SHA-1024** tùy chỉnh, đảm bảo đầu ra đủ lớn để tránh đụng độ có nghĩa với độ bảo mật $r \approx 2^{512}$.

5.2. Chữ ký Schnorr

5.2.1. Lý thuyết

Schnorr là sơ đồ chữ ký Sigma-protocol dựa trên giả thuyết logarit rời rạc (DL). Nó được ưa chuộng vì độ đơn giản của chứng minh bảo mật (trong mô hình Oracle ngẫu nhiên) và hỗ trợ tổng hợp tự nhiên.

Ký: Cho khóa riêng $d \in [1, r-1]$ và thông điệp m: 1. Chọn ngẫu nhiên $k \xleftarrow{R} [1, r-1]$ 2. Tính điểm $R = [k]G$ 3. Tính thách thức $e = H(R_x \| R_y \| m) \pmod r$ 4. Tính $s = k + e \cdot d \pmod r$ 5. Chữ ký: $\sigma = (R, s)$

Xác minh: Cho khóa công khai Q và chữ ký (R, s): 1. Tính $e = H(R_x \| R_y \| m)$ 2. Kiểm tra $[s]G = R + [e]Q$

Tính đúng đắn: $[s]G = [k + ed]G = [k]G + [e][d]G = R + [e]Q$ [ok]

5.2.2. Cấu trúc dữ liệu

```
pub struct SchnorrSignature<C: SWCurveConfig> {
    pub r_point: AffinePoint<C>,  // R = [k]G  (điểm 256 byte: 2x128 byte tọa
    pub s:       U1024,           // s in [0, r-1], 128 byte
}
```

Kích thước chữ ký: $2 \times 128 + 128 = \mathbf{384}$ byte (R gồm hai tọa độ x, y mỗi tọa độ 128 byte, cộng thêm s 128 byte).

5.2.3. Cài đặt

```
pub fn sign(private_key: &U1024, message: &[u8]) -> Self {
    let k = U1024::rand(&C::ORDER);
    let r_point = C::generator().mul(&k);
    let e       = Self::challenge_hash(&r_point, message); // H(Rx||Ry||m) mod
    let s       = (k_field + e * d_field).to_u1024();      // s = k + e.d mod r
    Self { r_point, s }
}

pub fn verify(&self, public_key: &AffinePoint<C>, message: &[u8]) -> bool {
    let e  = Self::challenge_hash(&self.r_point, message);
    let v1 = C::generator().mul(&self.s);                 // [s]G
    let v2 = self.r_point.add(&public_key.mul(&e.to_u1024())); // R + [e]Q
    v1 == v2
}
```

Hàm `challenge_hash` nối R_x, R_y, m rồi gọi `hash_message`, đảm bảo thách thức phụ thuộc vào toàn bộ ngữ cảnh — tránh tấn công forgery kiểu Pohlig-Hellman trên nonce.

5.2.4. Bảo mật

- **Unforgeability (EUF-CMA):** Chứng minh được trong Random Oracle Model dưới giả thuyết DL. Kẻ tấn công cần giải ECDLP hoặc bẻ hàm băm.
- **Nguy cơ nonce tái sử dụng:** Nếu k bị tái sử dụng cho hai thông điệp khác nhau, kẻ công phá có thể tính $d = (s_1 - s_2)/(e_1 - e_2) \pmod r$. Cài đặt dùng CSPRNG (`rand::thread_rng`) cho mỗi lần ký.

5.3. Chữ ký ECDSA

5.3.1. Lý thuyết

ECDSA (Elliptic Curve Digital Signature Algorithm) là chuẩn IEEE P1363 và FIPS 186-4. Nó khác Schnorr ở chỗ r trong chữ ký là **tọa độ** x **của điểm** R lấy modulo n, thay vì bản thân điểm R.

Ký: Cho khóa riêng d và thông điệp m: 1. $k \xleftarrow{R} [1, r-1]$ 2. $R = [k]G$ 3. $r_{\text{sig}} = R_x \pmod{n}$ (nếu $r_{\text{sig}} = 0$, thử lại) 4. $e = H(m)$ 5. $s = k^{-1}(e + r_{\text{sig}} \cdot d) \pmod{n}$ (nếu $s = 0$, thử lại) 6. Chữ ký: $\sigma = (r_{\text{sig}}, s)$

Xác minh: 1. Kiểm tra $r_{\text{sig}}, s \in [1, n-1]$ 2. $e = H(m)$, $w = s^{-1} \pmod{n}$ 3. $u_1 = ew$, $u_2 = r_{\text{sig}} \cdot w$, cả hai tính mod n 4. $P = [u_1]G + [u_2]Q$ 5. Hợp lệ khi $P \neq \mathcal{O}$ và $P_x \pmod{n} = r_{\text{sig}}$

5.3.2. Cấu trúc dữ liệu

```
pub struct EcdsaSignature {
    pub r: U1024,  // r_sig = R.x mod n, 128 byte
    pub s: U1024,  // s in [1, n-1],     128 byte
}
```

Kích thước chữ ký: $128 + 128 = \mathbf{256}$ byte — nhỏ hơn Schnorr vì không lưu điểm R đầy đủ.

5.3.3. Cài đặt

```
pub fn sign<C: SWCurveConfig>(private_key: &U1024, message: &[u8]) -> Self {
    let k     = U1024::rand(&C::ORDER);
    let r_pt  = C::generator().mul(&k);
    let e     = FieldElem::<C::ScalarField>::new(hash_message(message));
    let r     = FieldElem::<C::ScalarField>::new(r_pt.x.to_u1024());
    let k_inv = FieldElem::<C::ScalarField>::new(k).inv();
    let d     = FieldElem::<C::ScalarField>::new(*private_key);
    let s     = k_inv * (e + r * d);
    Self { r: r.to_u1024(), s: s.to_u1024() }
}

pub fn verify<C: SWCurveConfig>(&self, public_key: &AffinePoint<C>, message: &
    // Kiểm tra r, s in [1, n-1]
    if self.r.is_zero() || self.s.is_zero() { return false; }
```

```
    if self.r >= C::ORDER || self.s >= C::ORDER { return false; }

    let e = FieldElem::new(hash_message(message));
    let w = FieldElem::new(self.s).inv();       // w = s^{-1}
    let u1 = (e * w).to_u1024();
    let u2 = (FieldElem::new(self.r) * w).to_u1024();

    let p = C::generator().mul(&u1).add(&public_key.mul(&u2));
    !p.is_infinite && FieldElem::<C::ScalarField>::new(p.x.to_u1024()).to_u1024
}
```

5.3.4. So sánh Schnorr và ECDSA

Tiêu chí	Schnorr	ECDSA
Chuẩn hóa	ISO/IEC 14888-3	FIPS 186-4, IEEE P1363
Kích thước chữ ký	384 byte ($2 \times 128 + 128$)	256 byte (2×128)
Phép nhân vô hướng ký	1	1
Phép nhân vô hướng xác minh	2 (tuần tự)	2 (tuần tự)
Bảo mật chứng minh	RO Model, DL	Heuristic, DL + hash
Hỗ trợ tổng hợp	Có (MuSig, FROST)	Không
Rủi ro nonce tái sử dụng	Lộ khóa riêng	Lộ khóa riêng

5.4. Chữ ký tổng hợp BLS

5.4.1. Lý thuyết (tổng quan)

Chữ ký BLS (Boneh-Lynn-Shacham, 2001) là sơ đồ **tổng hợp phi tương tác**: n người ký với n khóa riêng khác nhau có thể tạo ra **một chữ ký duy nhất** đại diện cho tất cả, và xác minh chỉ cần một phép ghép cặp thay vì n phép.

Nền tảng: Dùng phép ghép cặp song tuyến $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ với: - $\mathbb{G}_1 = E(\mathbb{F}_p)$, $\mathbb{G}_2 = E'(\mathbb{F}_{p^2})$ (hoặc tương đương) - $\mathbb{G}_T \subset \mathbb{F}_{p^k}^*$

Ký: Người thứ i với khóa riêng d_i:

$$\sigma_i = [d_i]H(m) \in \mathbb{G}_1$$

trong đó $H : \{0,1\}^* \to \mathbb{G}_1$ là hàm băm lên nhóm điểm.

Tổng hợp: $\sigma = \sigma_1 + \sigma_2 + \cdots + \sigma_n$ (phép cộng điểm, không cần tương tác)

Xác minh tổng hợp:

$$e(\sigma, G_2) = e(H(m), Q_1 + Q_2 + \cdots + Q_n)$$

trong đó $Q_i = [d_i]G_2$ là khóa công khai của người i.

Tính đúng đắn:

$$e\left(\sum_i [d_i]H(m),\, G_2\right) = e\left(H(m),\, \sum_i [d_i]G_2\right) = e(H(m), \sum Q_i)\checkmark$$

5.4.2. Vai trò của đường cong KSS18

Đường cong được xây dựng trong luận văn này (họ Kachisa-Schaefer-Scott với $k = 18$) cung cấp nền tảng để cài đặt BLS:

- $\mathbb{G}_1 \subset E(\mathbb{F}_p)$: nhóm điểm trên trường cơ sở, kích thước r.
- $\mathbb{G}_T \subset \mathbb{F}_{p^{18}}^*$: nhóm đích của phép ghép cặp, bậc r, kích thước trường 18432 bit — đủ để kháng cả NFS lẫn TNFS.

Phép ghép cặp Optimal Ate (trên KSS18) chạy vòng lặp Miller với $O(\log r)$ bước, mỗi bước thực hiện phép tính trong $\mathbb{F}_{p^{18}}$. Cấu trúc NTT-friendly của p (two-adicity$(p-1) \geq 32$) giúp tăng tốc các phép nhân trong tháp trường.

Lưu ý: Cài đặt phép ghép cặp đầy đủ (vòng lặp Miller + phép lũy thừa lần cuối) nằm ngoài phạm vi của luận văn này và là hướng phát triển tiếp theo. Chương này tập trung vào Schnorr và ECDSA trên $\mathbb{G}_1$.

5.5. Cài đặt hệ thống ký (`curve1024-sig`)

Binary `curve1024-sig` cung cấp giao diện dòng lệnh kiểu GPG cho toàn bộ vòng đời khóa:

```
curve1024-sig keygen [-o <file>]
curve1024-sig sign   -f <file> [-k <key>] [--scheme schnorr|ecdsa]
curve1024-sig verify -f <file> [-k <pub>]
curve1024-sig export-pub [-k <priv>] [-o <pub>]
```

Các tham số được đọc từ `config/curve1024.toml` thông qua `build.rs` để sinh hằng số tĩnh — đảm bảo zero-cost abstraction và không có chi phí khởi tạo runtime.

CHƯƠNG 6
CÀI ĐẶT VÀ ĐÁNH GIÁ

Chương này đánh giá toàn diện hệ thống đã xây dựng, không chỉ dừng lại ở trình bày số liệu mà tập trung **chứng minh giá trị** của hệ thống trên hai phương diện: hiệu năng thực tế và độ an toàn mật mã. Mục tiêu trả lời câu hỏi: *Các con số đo được nói lên điều gì? Hệ thống này có an toàn không? Có dùng được trong thực tế không?*

6.0.1. Đánh giá hiệu năng

6.0.1.1. Kết quả đo đạc

Toàn bộ hệ thống được đo bằng framework Criterion (release build, tối ưu LLVM O3, 10 mẫu/phép đo, phần cứng thông thường không có AVX-512). Kết quả cụ thể như sau:

Phép đo	Min	Mean	Max
`KeyPair::generate`	1.629 s	**1.687 s**	1.757 s
`Schnorr/sign`	1.473 s	**1.551 s**	1.641 s
`Schnorr/verify`	2.656 s	**2.831 s**	3.025 s
`ECDSA/sign`	1.524 s	**1.606 s**	1.711 s
`ECDSA/verify`	2.847 s	**3.140 s**	3.459 s
Nhân vô hướng $[k]G$	1.061 s	**~1.1–1.3 s**	1.489 s

6.0.1.2. Phân tích: Kết quả có hợp lý về mặt lý thuyết không?

Trước tiên cần nhìn nhận rằng Curve1024 sử dụng trường nguyên tố p **1024-bit** — gấp 4 lần kích thước của secp256k1 (256-bit) hay P-384. Đây là lựa chọn chủ động, không phải sai sót thiết kế, vì mục tiêu là bảo mật 256-bit và tính pairing-friendly. Hệ quả tất yếu là hiệu năng chậm hơn đáng kể.

Phép nhân vô hướng dùng thuật toán Double-and-Add với **1024 vòng lặp** (tương ứng kích thước bit của trường vô hướng $r = 512$ bit — số vòng lặp bằng độ dài bit của *trường cơ sở* p). Mỗi vòng lặp thực hiện một phép `double` và có thể một phép `add`, mỗi phép toán đó cần **1 phép nghịch đảo** modulo p trong hệ tọa độ Affine hiện tại. Phép

nghịch đảo được tính bằng định lý Fermat: $a^{-1} = a^{p-2} \pmod p$, tức là gọi đệ quy `pow` với khoảng **~1536 phép nhân Montgomery** 1024-bit bên trong. Đây là nguồn gốc của con số ~1.1–1.6 giây/phép nhân vô hướng.

Schnorr ký nhanh hơn ECDSA ~3% do Schnorr không cần tính nghịch đảo $k^{-1} \pmod r$ trong bước sinh chữ ký. Xác minh (cả Schnorr và ECDSA) tốn gần gấp đôi ký do cần **2 phép nhân vô hướng độc lập** ($[s]G$ và $[e]Q$ cho Schnorr; $[u_1]G$ và $[u_2]Q$ cho ECDSA). Tỷ lệ này hoàn toàn phù hợp với dự kiến lý thuyết.

So với secp256k1, độ chênh lệch (~1.5 s so với ~0.2 ms) xuất phát chủ yếu từ ba yếu tố tích lũy: (1) scalar dài gấp 4 lần kéo dài thêm 4× số vòng lặp, (2) mỗi phép nhân Montgomery trên số 1024-bit tốn ~16 lần công sức so với 256-bit (chi phí $O(n^2)$ với n số limb), và (3) secp256k1 có thư viện tối ưu nhiều năm với assembly AVX-512 được tinh chỉnh. Kết hợp lại, khoảng cách ~8000× là kỳ vọng hợp lý, không phải dấu hiệu của thuật toán sai.

6.0.1.3. Tính khả thi trong ứng dụng thực tế

Với thời gian ~1.5–1.7 giây cho một phép ký, hệ thống hoàn toàn phù hợp với các **kịch bản ký ngoại tuyến** như ký tài liệu pháp lý, ký gói phần mềm khi phát hành, ký giao dịch tài chính giá trị lớn, hay ký chứng chỉ trong hạ tầng PKI — những trường hợp mà độ trễ vài giây là chấp nhận được và bù đắp hoàn toàn bởi biên bảo mật 256-bit so với 128-bit của các chuẩn thông thường.

Ngược lại, hệ thống **chưa phù hợp** cho các ứng dụng yêu cầu độ trễ dưới mili-giây như TLS handshake xử lý hàng nghìn kết nối/giây, giao dịch thanh toán tốc độ cao, hay ký batch thời gian thực. Đây là sự đánh đổi có chủ ý — tập trung vào tính đúng đắn và bảo mật của nền tảng toán học trước, tối ưu hóa sau.

Nhờ áp dụng **phép nhân Montgomery** thay cho phép chia lấy dư thông thường, tất cả phép nhân trong $\mathbb{F}_p$ đều thực thi bằng phép shift và trừ có điều kiện, không cần chia số 1024-bit cho p — một phép toán đắt hơn nhân ~5–10 lần. Đây là tối ưu quan trọng nhất đã được triển khai trong cài đặt hiện tại.

Các hướng cải tiến hiệu năng tiếp theo được phân tích tại §5.4.

6.0.2. Đánh giá độ an toàn

6.0.2.1. An toàn trước tấn công cổ điển

Hệ thống sử dụng trường nguyên tố p có độ lớn 1024-bit và bậc nhóm r là 512-bit. Bài toán ECDLP trên nhóm $E(\mathbb{F}_p)$ — tức là tìm k từ $Q = [k]G$ — hiện chưa có thuật

toán tốt hơn thuật toán tổng quát. Thuật toán tốt nhất hiện tại là **Pollard's** ρ, có độ phức tạp $O(\sqrt{r}) \approx O(2^{256})$ phép toán nhóm. Với 2^{256} bước tính toán cần thực hiện, ngay cả khi huy động toàn bộ năng lực tính toán của nhân loại — mỗi nguyên tử Trái Đất thực hiện 10^9 phép tính mỗi giây từ khi Big Bang — vẫn không đủ để phá vỡ khóa trong tuổi thọ của vũ trụ. Đây là **biên an toàn tuyệt đối** trước mọi hệ thống máy tính cổ điển.

Mức bảo mật 256-bit vượt xa chuẩn tối thiểu NIST khuyến nghị (128-bit cho các ứng dụng đến năm 2030) và tương đương với AES-256 — chuẩn mã hóa đối xứng mạnh nhất hiện nay. Đây là mục tiêu tường minh của thuật toán Cocks-Pinch cải tiến được trình bày tại Chương 3.

6.0.2.2. An toàn trước tấn công Invalid Curve

Invalid Curve Attack là một dạng tấn công nguy hiểm trong ECC: kẻ tấn công gửi một điểm P' *không thuộc đường cong* E nhưng vẫn hợp lệ về mặt định dạng, ép hệ thống tính toán trên một đường cong suy biến có bậc nhỏ, từ đó thu thập thông tin về khóa riêng qua nhiều truy vấn.

Hệ thống hiện tại **miễn nhiễm hoàn toàn** với tấn công này. Mọi điểm được tạo ra thông qua `AffinePoint::new(x, y)` đều tự động kiểm tra điều kiện:

$$y^2 \equiv x^3 + b \pmod{p}$$

bằng lời gọi `assert!(self.is_on_curve())` ngay trong hàm khởi tạo. Không tồn tại đường dẫn code nào cho phép một điểm không hợp lệ tồn tại trong hệ thống. Điều này được xác nhận bởi test `test_curve_point` trong bộ kiểm thử đơn vị.

6.0.2.3. An toàn trước tấn công MOV

Tấn công MOV (Menezes-Okamoto-Vanstone) sử dụng phép ghép cặp (pairing) để ánh xạ bài toán ECDLP trong $E(\mathbb{F}_p)$ về bài toán DLP trong trường hữu hạn mở rộng $\mathbb{F}_{p^k}$, nơi bài toán DLP có thể được giải hiệu quả hơn bằng thuật toán chỉ số (index calculus).

Tuy nhiên, hiệu quả của tấn công này phụ thuộc vào việc $\mathbb{F}_{p^k}$ có đủ nhỏ để bài toán DLP khả thi hay không. Với Curve1024, bậc nhúng $k = 18$, trường mở rộng $\mathbb{F}_{p^{18}}$ có kích thước $|p^{18}| = 18 \times 1024 = 18432$ bit. Bài toán DLP trong trường 18432-bit hoàn toàn không thể giải được với công nghệ hiện tại — trên thực tế, kinh nghiệm cộng đồng yêu cầu $k|p| \geq 3072$ bit để kháng tấn công này ở mức 128-bit; hệ thống đạt $18 \times 1024 = 18432$

bit, vượt xa tiêu chuẩn này. Kết quả này được test `test_mov_attack_resistance` xác nhận tường minh.

6.0.2.4. An toàn trước tấn công Anomalous

Tấn công Anomalous (SSSA) khai thác trường hợp đặc biệt khi $\#E(\mathbb{F}_p) = p$, tức là số điểm trên đường cong bằng đúng modulus. Trong trường hợp này, tồn tại một đẳng cấu nhóm ánh xạ bài toán ECDLP về bài toán trên nhóm cộng $\mathbb{Z}/p\mathbb{Z}$, cho phép giải trong thời gian tuyến tính.

Curve1024 có $\#E(\mathbb{F}_p) = h \cdot r$ với r là bậc nhóm con nguyên tố 512-bit và h là cofactor, trong khi p là số nguyên tố 1024-bit. Hiển nhiên $\#E(\mathbb{F}_p) \neq p$ vì $|r| = 512 < 1024 = |p|$, nên điều kiện anomalous không thoả mãn. Test `test_anomalous_attack_resistance` xác minh điều này.

6.0.2.5. Hạn chế: Timing Side-Channel Attack

Đây là điểm cần trung thực học thuật: thuật toán **Double-and-Add** hiện tại, mặc dù có `conditional_select` ở từng bước đơn lẻ, vẫn có tổng số phép `add` thực hiện **phụ thuộc vào số lượng bit 1 trong scalar** k. Kẻ tấn công có thể đo thời gian thực thi để suy luận về trọng số Hamming của khóa riêng, từ đó thu hẹp không gian tìm kiếm.

Đây là sự đánh đổi có chủ ý để giữ kiến trúc đơn giản (nguyên tắc KISS — Keep It Simple and Secure). Biện pháp khắc phục là thay thế bằng **Montgomery Ladder**, thuật toán thực hiện đúng 1024 phép `double` và 1024 phép `add` bất kể giá trị scalar, đảm bảo tính *constant-time* thực sự. Đây là hướng phát triển ưu tiên cao.

Toàn bộ kết quả kiểm thử **36/36 test PASSED** (17 đơn vị + 19 tích hợp) xác nhận tính đúng đắn toán học và an toàn của hệ thống trên mọi tiêu chí đã mô tả.

6.0.3. Kết luận và hướng phát triển

6.0.3.1. Kết quả đạt được

Luận văn đã hoàn thành xây dựng một hệ thống mật mã đường cong elliptic hoàn chỉnh từ nền tảng toán học đến ứng dụng thực tế:

- **Cơ sở toán học vững chắc:** Cài đặt $\mathbb{F}_p$ với phép nhân Montgomery, toán học điểm affine đầy đủ, và phép nhân vô hướng thuần Rust không phụ thuộc thư viện ngoài.
- **Đường cong đặc biệt:** Sinh được đường cong pairing-friendly 1024-bit ($k = 18$, bảo mật 256-bit, NTT-friendly trên cả hai trường) bằng thuật toán Cocks-Pinch cải tiến — một đóng góp gốc của luận văn.

- **Sơ đồ chữ ký hoàn chỉnh:** Schnorr và ECDSA hoạt động chính xác và được kiểm thử đầy đủ.
- **Bộ kiểm thử toàn diện:** 36 test bao phủ từ số học cấp thấp đến bảo mật tầng cao, kể cả mô phỏng tấn công MOV và Anomalous.

6.0.3.2. Hướng phát triển

Tối ưu hóa hiệu năng:

1. **Tọa độ Jacobian/Projective:** Loại bỏ phép nghịch đảo trong mỗi bước `add`/`double` bằng cách làm việc trong hệ tọa độ $(X : Y : Z)$ thay vì (x, y) Affine. Toàn bộ vòng lặp nhân vô hướng chỉ cần 1 phép nghịch đảo duy nhất ở bước chuyển về Affine cuối cùng — ước tính giảm thời gian ký xuống dưới 200 ms.

2. **Sliding Window / NAF:** Giảm ~25–30% số phép `add` trong nhân vô hướng so với Double-and-Add cơ bản. Kết hợp với Jacobian, thời gian ký dự kiến đạt dưới 150 ms.

3. **AVX-512 / SIMD assembly:** Vector hóa các phép nhân 64×64-bit limb trong Montgomery multiplication trên kiến trúc x86-64 hiện đại. Các thư viện như `blst` (Ethereum) đạt tốc độ BLS12-381 signing dưới 1 ms nhờ kỹ thuật này.

Bảo mật cấp cao:

4. **Montgomery Ladder:** Thay thế Double-and-Add bằng Montgomery Ladder để đạt *constant-time* thực sự, loại bỏ timing side-channel theo giá trị scalar.

5. **Phép ghép cặp (Pairing):** Cài đặt Miller loop và bước mũ cuối (final exponentiation) để kích hoạt đầy đủ BLS aggregate signatures, mở ra ứng dụng trong các giao thức đồng thuận phân tán.

6. **Multi-Scalar Multiplication (MSM — Pippenger):** Tối ưu bước xác minh $[s]G + [e]Q$ bằng thuật toán Pippenger, giảm ~50% công việc so với hai phép nhân tuần tự.

Các hướng 1 và 4 là ưu tiên cao nhất vì cùng lúc cải thiện cả hiệu năng và bảo mật, không yêu cầu thay đổi kiến trúc cấp cao.

CHƯƠNG 7
KẾT LUẬN

7.0.1. Tổng kết

Tóm tắt những gì đã làm được trong khóa luận…

7.0.2. Hạn chế

Những hạn chế của hệ thống…

7.0.3. Hướng phát triển

Đề xuất các hướng phát triển trong tương lai…

TÀI LIỆU THAM KHẢO